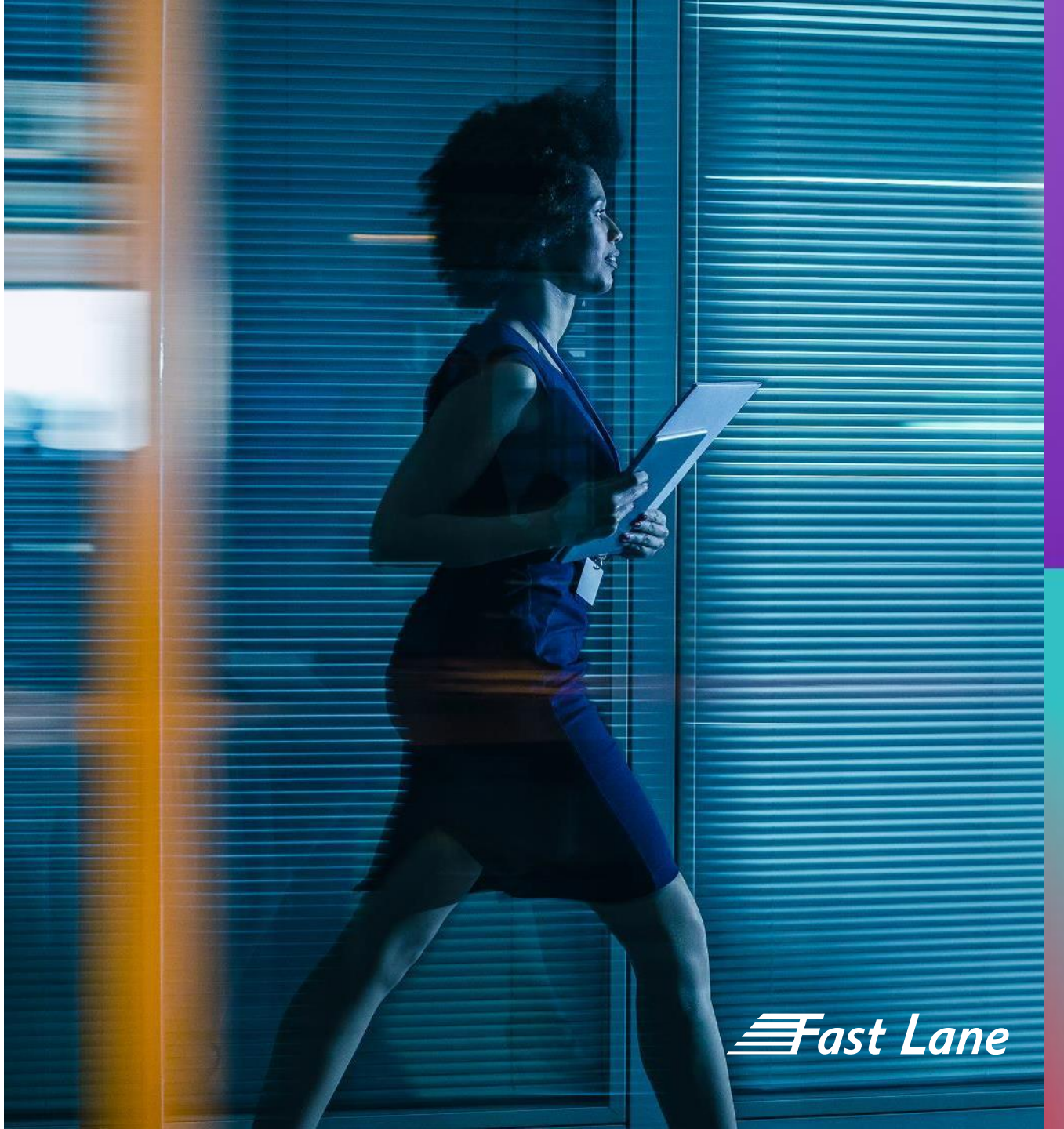
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

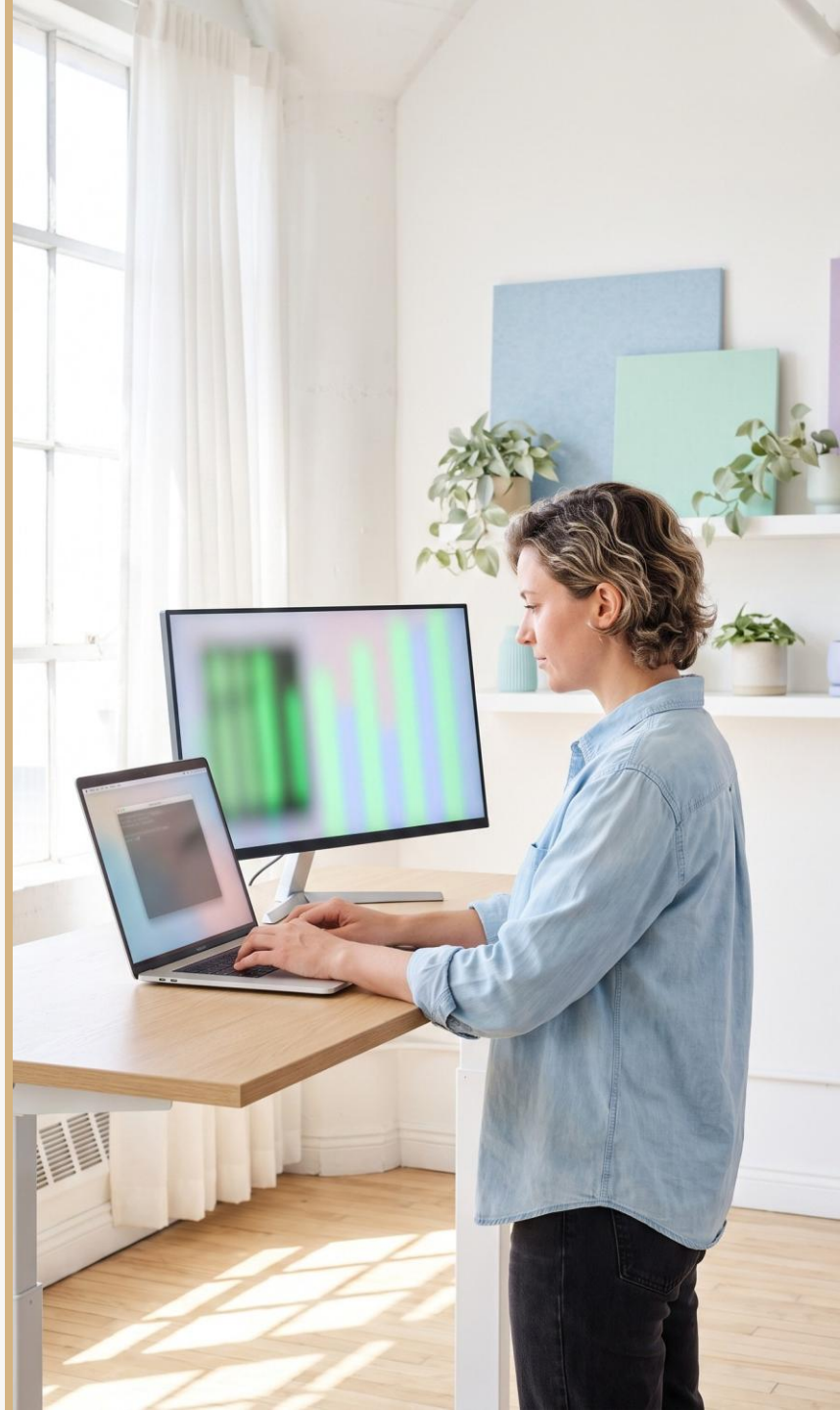
SCAN TO CONNECT
integrations.at

MODULE 5

Automate Claude Code from the Command Line

Headless runs, CI gates, parallel lanes and goal loops

Agentic Software Engineering with Claude Code



01

Introduction

Headless runs, JSON output and tool scope

03

CI Gates

Sentinel tokens that pass or block a build

04

Parallel Worktrees and Policy Hooks

One lane per agent, one gate for all

05

Always-On and Goal Patterns

Loops that stop on a checkable goal

Automating Claude Code from the command line

Headless runs, CI gates, parallel lanes and goal loops

- **Headless mode**: `claude -p` with JSON output, exit codes and scoped tools
- **Gates and lanes**: sentinel tokens in CI, worktrees under one policy hook
- **Goal loops**: agents that keep working until a checkable condition holds



Overview

The path through the module

- Start with **headless mode**: `claude -p`, the JSON envelope and tool scoping
- Turn an audit into a build gate with **sentinel tokens**
- Run parallel agents in **git worktrees** under one policy hook
- Finish with **goal loops** bounded by `--max-turns`



Where headless mode fits

Work you can describe completely up front

- **Pull-request gates**, parallel feature lanes and batch work across a repo
- **Scheduled maintenance**: issue triage, changelog refresh, dead links
- **Goal-driven repair** and JSON glue inside larger scripts
- State the success condition, output format and boundaries in the prompt
- Keep work interactive when it needs judgement you cannot pre-state



01

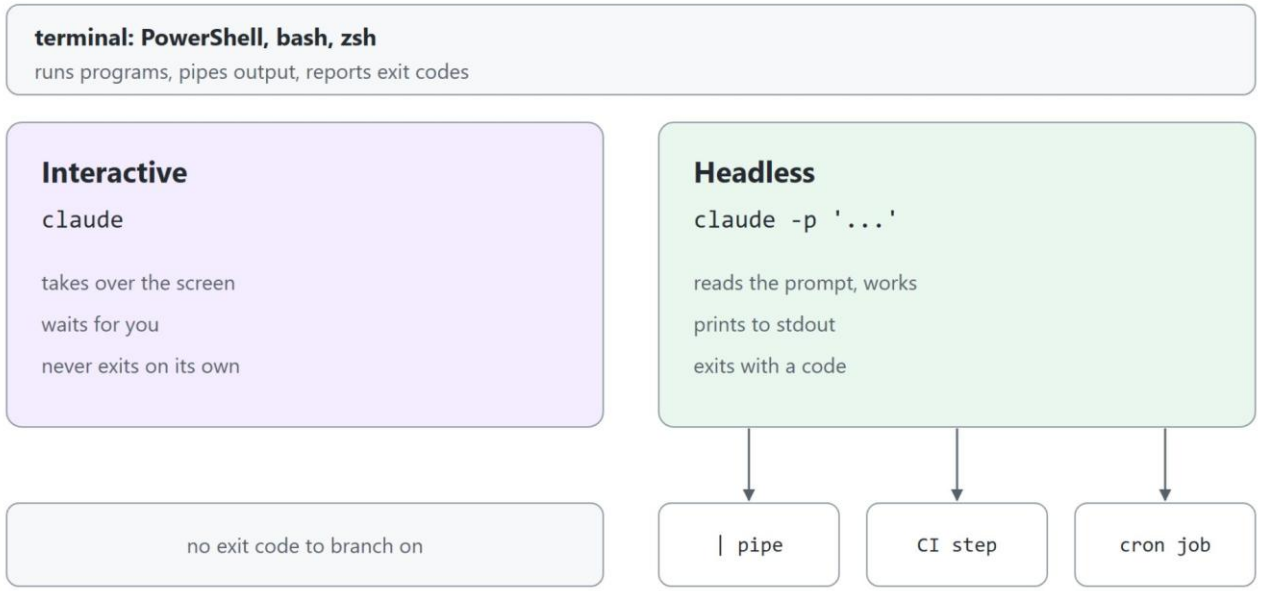
Introduction

Headless runs, JSON output and tool scope

Interactive and headless mode

Only the -p half composes with the terminal

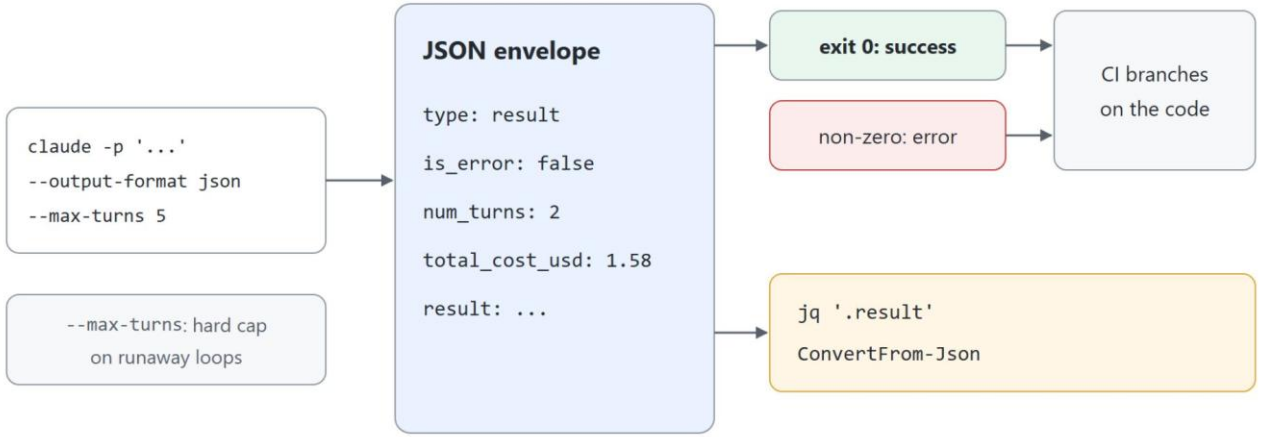
- The **terminal** runs programs, pipes output and reports exit codes
- `claude` is one program inside it, with two modes
- **Interactive** `claude` owns the screen and never exits on its own
- **Headless** `claude -p` prints to stdout and exits with a code
- Only headless runs inside pipes, CI steps and cron jobs



Headless output and exit codes

A JSON envelope a script can branch on

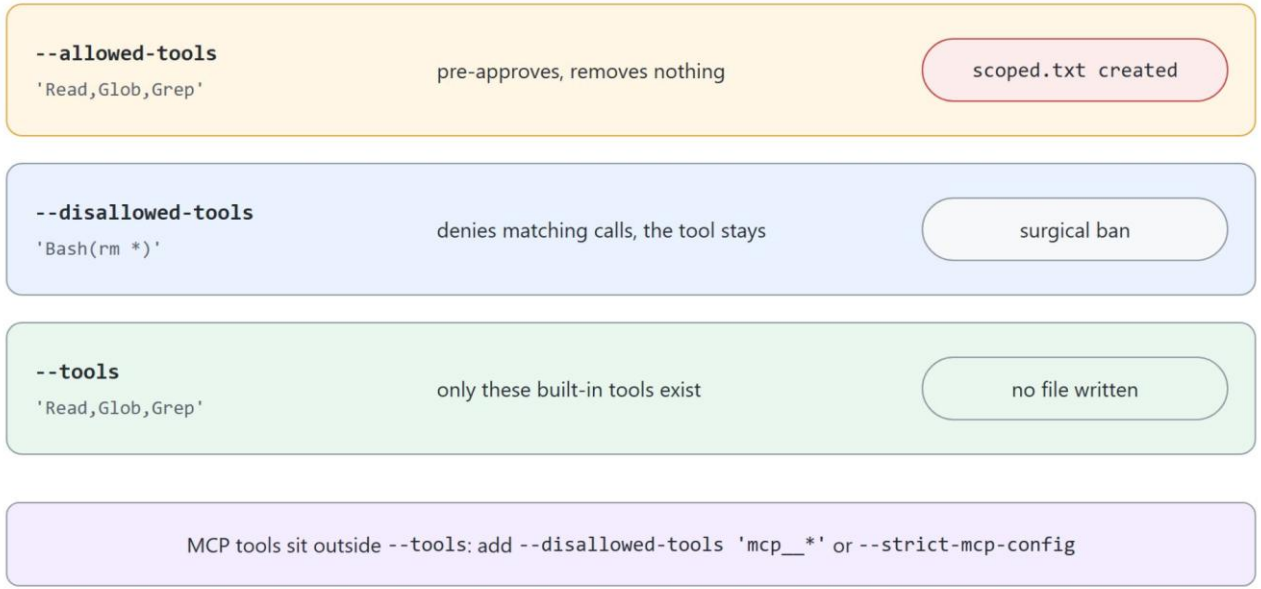
- `--output-format json` wraps the run in a **JSON envelope**
- Fields to branch on: `is_error`, `result`, `num_turns`, `total_cost_usd`
- **Exit 0** on success, non-zero on error: the signal CI needs
- **`--max-turns`** is a hard cap against runaway loops
- Read fields with `jq` or `ConvertFrom-Json`, never by parsing prose



Tool scoping flags

Granting a permission is not removing a capability

- `--allowed-tools` pre-approves tools and removes nothing
- `--disallowed-tools` bans a tool or one pattern such as `Bash(rm *)`
- `--tools` replaces the built-in set: the only read-only run
- Deny `Write` but leave `Bash`, and a shell redirect still writes
- MCP tools need `--disallowed-tools "mcp__*"` or `--strict-mcp-config`



03

CI Gates

Sentinel tokens that pass or block a build

Sentinel tokens in a CI gate

Three outcomes: pass, fail, indeterminate

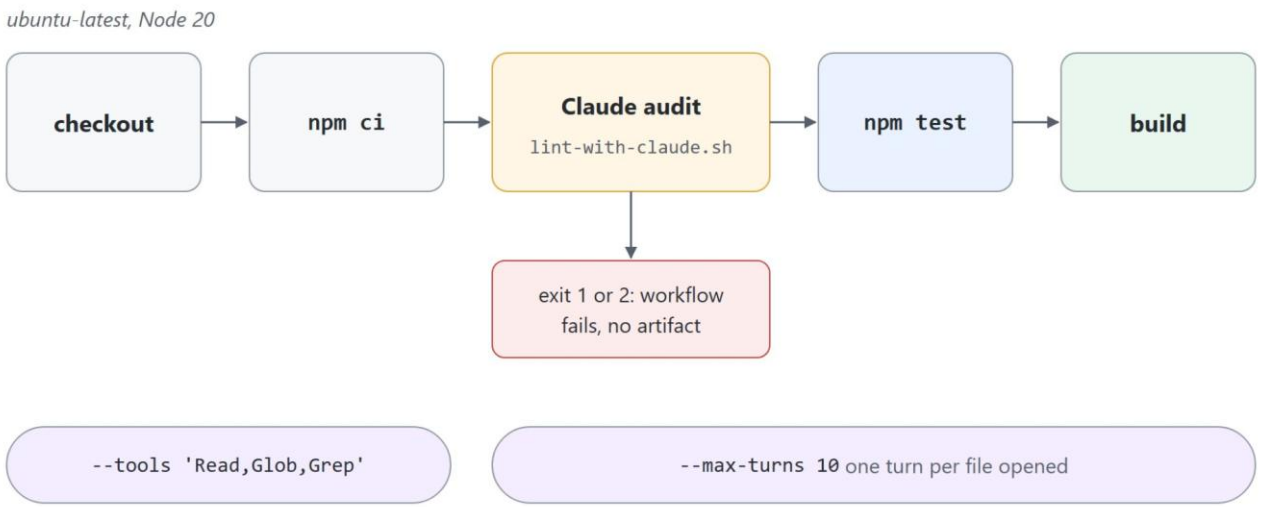
- The prompt ends every audit with `LINT_PASS` or `LINT_FAIL`
- The script greps the exact token, never the model's phrasing
- **Exit 0** pass, **exit 1** fail, **exit 2** for anything else
- Ambiguous output counts as neither pass nor fail
- PowerShell needs case-sensitive `-cmatch`, not `-match`



Wiring the gate into GitHub Actions

A read-only audit that runs before the build

- `--tools "Read,Glob,Grep"`: the audit physically cannot write
- Budget `--max-turns` per file opened: **10** for a four-file project
- Run the audit **before the build** so a failure blocks every artifact
- GitHub Actions, GitLab CI and Jenkins all act on the exit code
- The `.ps1` twin runs on `ubuntu-latest` with `shell: pwsh`



Proving the gate fails the build

Break slugify on purpose, read the exit code

- Baseline: `lint-with-claude.sh` exits 0 on the clean codebase
- Remove `trim()` from `slugify` and leave the test untouched
- Expected: **`LINT_FAIL`** in the output and **exit 1**
- Exit 2 instead: the grep pattern does not match the token's casing



04

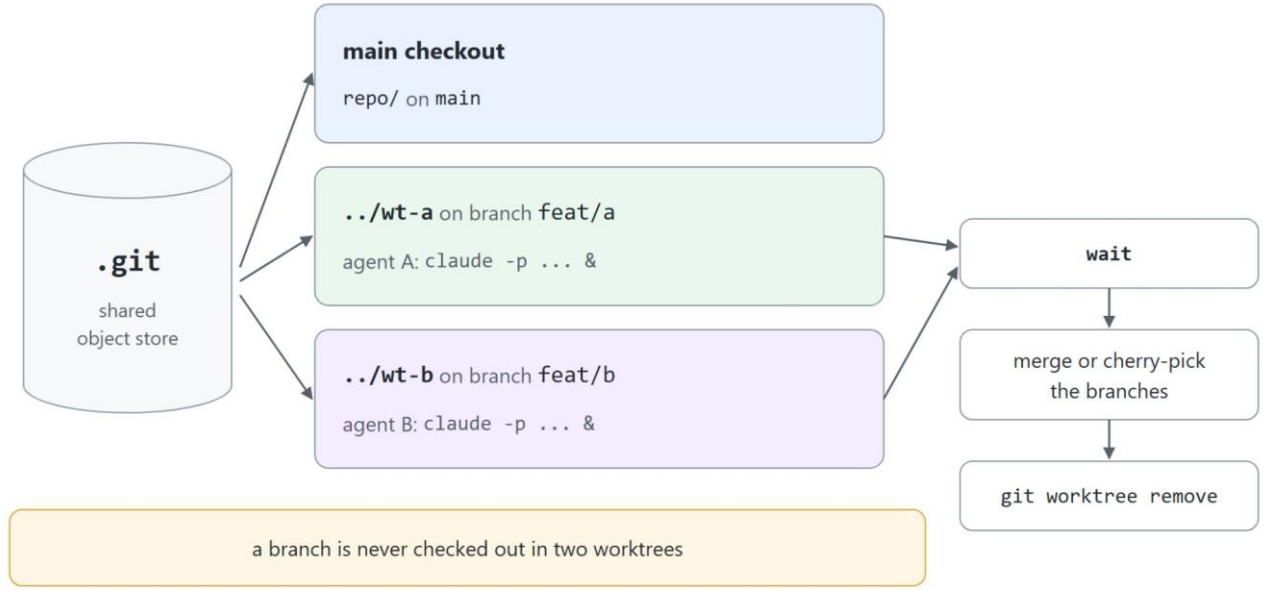
Parallel Worktrees and Policy Hooks

One lane per agent, one gate for all

Git worktrees as agent lanes

One repository, one checkout and branch per agent

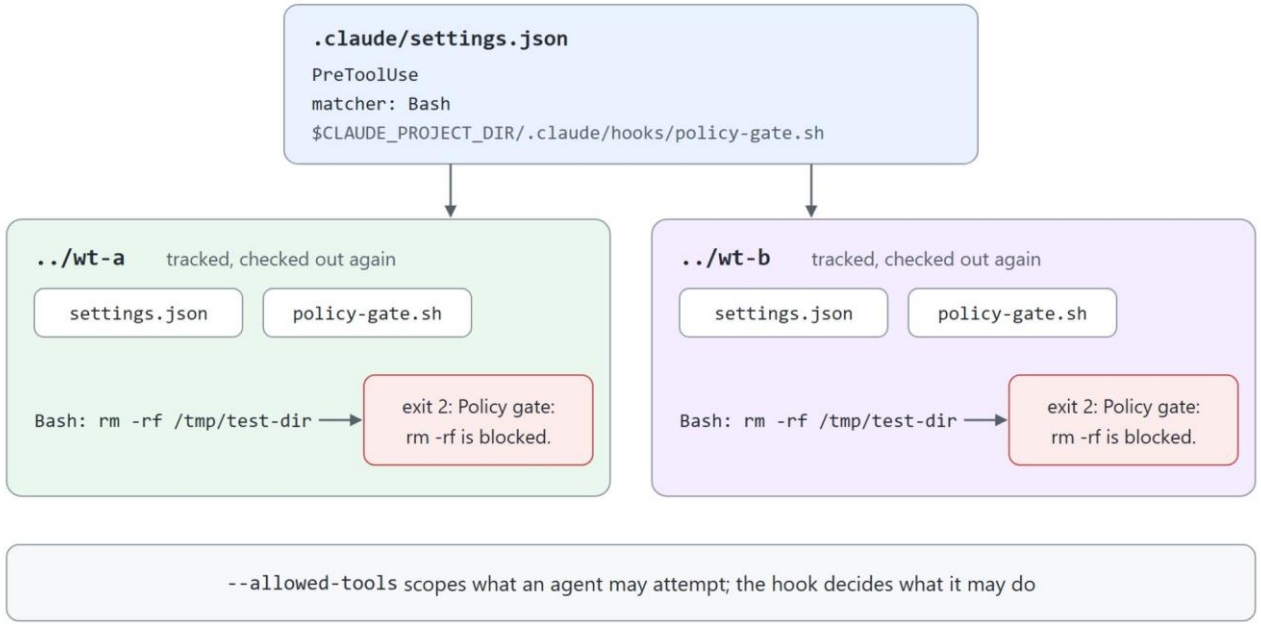
- `git worktree add <path> -b <branch>`: a separate checkout, not a clone
- One shared `.git` and object store: no network, no copy
- Each agent commits on its own branch and cannot reach the others
- **One branch per lane**: a branch is never checked out twice
- Launch lanes in the background, join with `wait`, then merge



A policy gate in every worktree

The hook travels with the repository

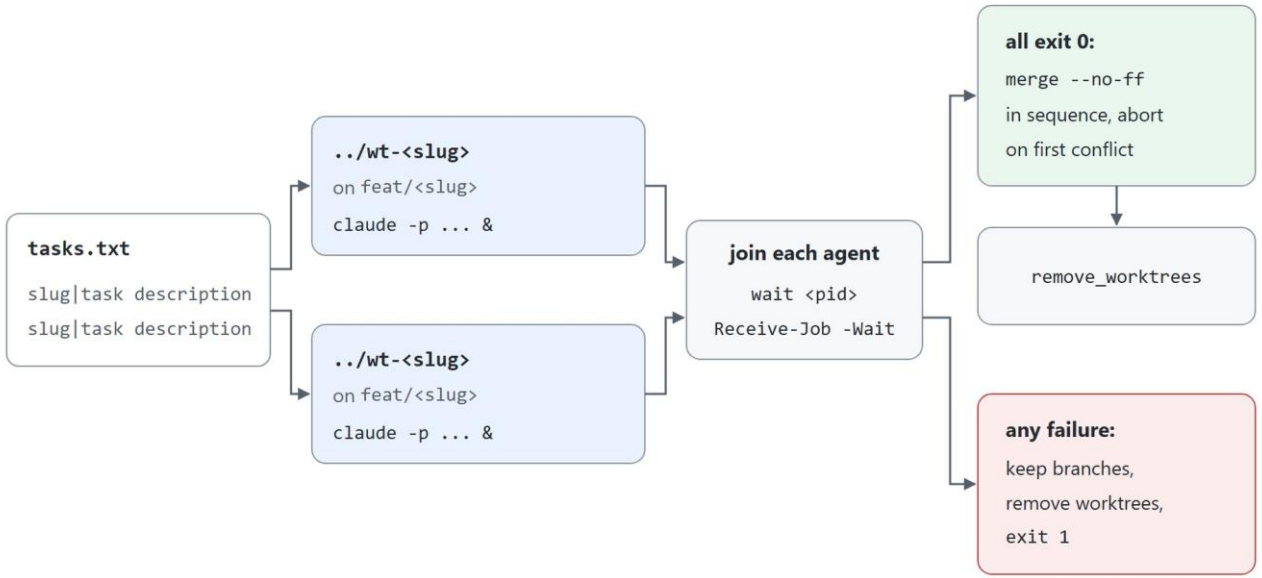
- A **`PreToolUse`** hook registered in the tracked `.claude/settings.json`
- `$CLAUDE_PROJECT_DIR` resolves to each worktree's own copy
- **Exit 2** blocks the call; any other non-zero code only warns
- Test the gate with a piped JSON payload before spending a turn
- `--allowed-tools` scopes what an agent tries, the hook what it can do



A task-queue orchestrator

One line becomes one worktree, branch, agent and log

- Input: `tasks.txt`, one `slug|task description` per line
- Each task gets `../wt-<slug>` on `feat/<slug>` and a background agent
- Capture **each** exit code: `wait <pid>` or the job's `$LASTEXITCODE`
- Merge with `--no-ff` only when every agent succeeded
- A failed run keeps its branches so the partial work stays inspectable



05

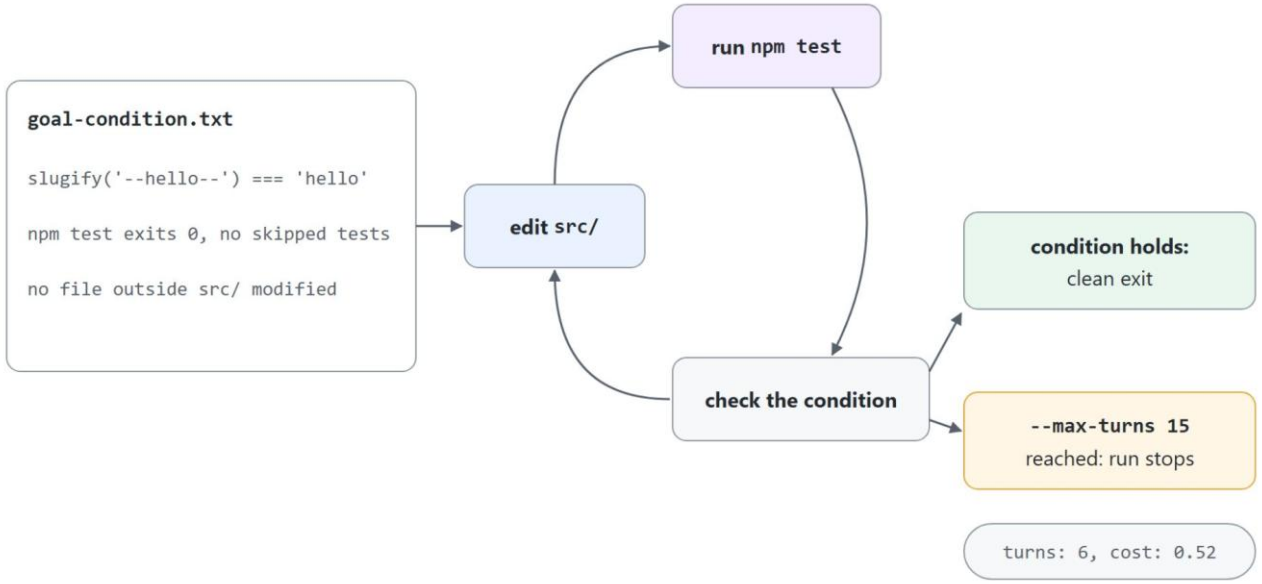
Always-On and Goal Patterns

Loops that stop on a checkable goal

The goal pattern

Loop until a checkable condition holds

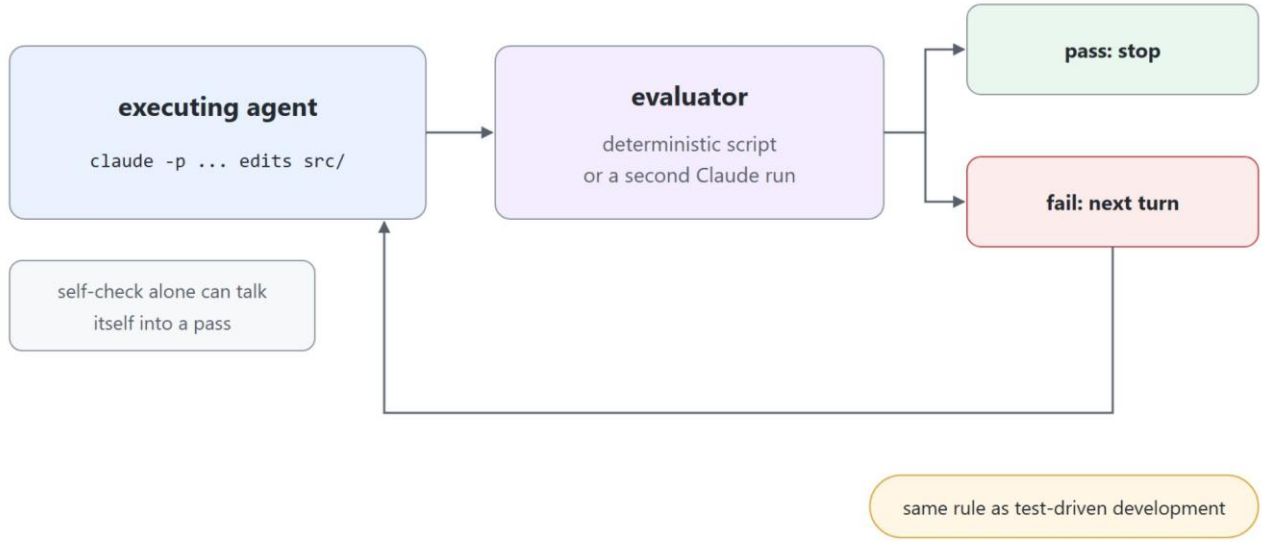
- A goal is **verifiable**: it resolves to a binary outcome
- "Fix the bug" is a wish; "`npm test` exits 0" is a condition
- The condition ships in `goal-condition.txt`, apart from the prompt
- Every turn is checked against it, and a pass ends the run
- **`--max-turns`** is the only guarantee that the loop ends



Separate execution from evaluation

The agent that writes the fix does not grade it

- A self-grading agent can convince itself that a failing condition passed
- A **secondary evaluator** checks the condition independently
- The evaluator is a **deterministic script** or a separate Claude run
- Same rule as test-driven development: the author is not the grader



Bounded autonomous runs

No prompts, a turn ceiling and a known worst case

- `--allowed-tools "Read,Edit,Bash(npm:*)"` pre-approves a narrow set
- `--permission-mode acceptEdits` applies edits without asking
- **`--max-turns`** stops loops whose goal can never be met
- Worst case is turns times cost: 15 turns at $0.10 is **$1.50**
- Read `result`, `num_turns` and `total_cost_usd` from the envelope



Takeaways

What to carry into the subagents module

- `claude -p` exits with a code: branch on it, never on prose
- Scope with `--tools`; `--allowed-tools` only removes prompts
- Sentinel tokens give a gate three outcomes: pass, fail, indeterminate
- One worktree and branch per agent, one hook for every lane
- State goals as checkable conditions and always cap `--max-turns`

